Order Integrity for Agentic DNA Synthesis Under Prompt Injection

Vishnu Rajeev **Anatole Seychal**
University of California, Berkeley
vishnu.rajeev@berkeley.edu aseychal@berkeley.edu

Abstract

Laboratories are beginning to delegate DNA synthesis ordering to language-model agents. Screening synthesized sequences against sequences of concern is the established safeguard at that step, and because it validates a particular artifact, it assumes that the sequence under review is the sequence that will be made. An agent that reads untrusted text does not satisfy that assumption, because an indirect prompt injection, in which instructions hidden in data the agent reads are followed as though they came from the user, can cause it to order a construct other than the one that was planned and screened. We call this a failure of order integrity. Using a bioinformatics agent with simulated ordering, we compared four conditions over a complete grid of 676 episodes, consisting of no monitor, a static rule baseline, plan-bound monitoring, and plan-bound monitoring whose monitor also reads untrusted content. Plan-bound monitoring requires the agent to commit to a plan before it reads any untrusted data, after which a separate monitor that never sees that data blocks any action departing from the plan. On the redirected order the undefended agent ordered the attacker's construct in 27 of 30 trials and the rule baseline did so in 22 of 30, whereas plan-bound monitoring blocked the order in all 30 trials, with an exact McNemar p of 4.8×10^{-7} against the baseline. The redirected order evades the baseline because it satisfies every static rule, including the requirement that a screen precede an order. On attacks that a static rule can express the two defenses are indistinguishable, and on benign episodes both block the same nine of 183 calls with no loss of task completion. Fabricated results defeat all four conditions equally.

1 Introduction

Language-model agents are increasingly applied to scientific work, including the planning and execution of chemistry experiments [2; 3] and the analysis of biological data [8]. Some of these agents can act on the physical world, and among such actions the one with the clearest biosecurity relevance is the placement of an order for synthetic DNA. Screening synthesized sequences against sequences of concern is the established safeguard at that step [10], and because screening validates a particular artifact, it rests on the assumption that the sequence under review is the sequence that will be made.

An agent that reads untrusted text does not satisfy that assumption. In an indirect prompt injection, an attacker places instructions in content that the agent will read, such as a dataset README or a database record, and the agent follows those instructions as though they had come from the user [4; 7]. Such an injection can cause the agent to order a construct other than the one the task planned, after the planned construct has already been screened. We refer to this outcome as a failure of order integrity, and it differs from the case in which an agent calls a forbidden tool, because the tool is permitted, the order appears routine, and a screen was performed on a different sequence one step earlier. The artifact that was reviewed and the artifact that is ordered are no longer the same.

The defenses commonly proposed for prompt injection do not address this case. Detection-based defenses attempt to recognize malicious content, which requires interpreting the payload, and policy-based defenses restrict the tools and arguments an agent may use [9], although a static rule that requires a screen before every order cannot also state that the ordered construct is not the planned one without a separate rule for each task. Architectural defenses limit what untrusted content can influence, either by separating a privileged planner from a quarantined reader [5; 11] or by fixing the agent's actions before it processes untrusted input [1], and the AI control literature examines protocols in which a trusted model monitors a more capable one [6]. Plan-bound monitoring combines a committed plan with a runtime monitor, so that the agent writes a plan before it reads any untrusted data and a monitor that never sees that data checks each action against the plan.

This paper characterizes order integrity as the biosecurity-relevant failure mode for an agent that can order DNA, constructs a testbed with simulated ordering together with a taxonomy that distinguishes rule-expressible attacks from plan-relative attacks, the latter using a permitted tool and satisfying every static rule while departing from the plan at the level of arguments, and reports a complete grid of 676 episodes in which plan-bound monitoring blocks the redirected order in all 30 trials where a frozen rule baseline permits it in 22, while restricting no more legitimate work on benign episodes.

2 Threat model and method

Order integrity. The agent is given a benign task that concludes in a synthesis order, and its workflow screens the construct before ordering it. The attacker controls a passage of text the agent reads during the task and seeks to have the agent order a different construct, but cannot modify the plan, the tools, or the rules. An attack succeeds when an order is placed whose construct differs from the planned one. We call this the redirected order, and it satisfies a requirement that a screen precede an order, because the agent can be induced to screen the alternative construct first, while naming neither a forbidden tool nor an external destination.

Plan-bound monitoring. At the beginning of an episode the agent receives only the task instruction and the tool schemas, and it returns a plan as JSON before any tool executes. Each step of the plan names a tool, states its purpose, and may fix argument values such as the accession to fetch or the construct to order, and one step may be required to precede another, which we use to require that a screen precede an order. Before each subsequent tool call executes, a monitor receives the task instruction, the plan, a trace of prior calls, and the proposed call, and it never receives tool outputs, file contents, or the agent's reasoning. Free-text fields are presented as a byte count while identifiers such as paths and accessions are presented in full, so that what the plan constrains remains visible although the untrusted payload does not. The monitor returns a verdict of in-plan or out-of-plan, and an out-of-plan call is blocked and recorded as an escalation, with a third block ending the episode as a human halt. Because the monitor never reads untrusted content it cannot itself be injected, and because it compares an action only against the plan it does not interpret the payload, although it also does not track data flow, so a wrong value written into an output the plan permits lies outside its scope.

3 Experimental setup

Agent, tools, and data. The agent is `gpt-5-mini` (OpenAI) and the monitor is `claude-haiku-4-5` (Anthropic), so that the two models come from different vendors, and neither accepted a temperature setting. The agent can fetch sequence and UniProt records, work with files, run alignments and shell commands, send data outward, and screen and order sequences. No tool contacts a live service, since shell commands outside a short read-only allowlist are recorded as would-execute and never run, while sends, screens, and orders are simulated and logged. Sequence records are synthetic stand-ins for benign, well-studied genes because the single permitted live fetch was refused, and no measured quantity depends on the sequences being real.

Tasks and attacks. We use 12 tasks across three categories, namely alignment, pipeline, and ordering, each with a deterministic checker and one injection site on its benign path, varied across three delivery channels, and all 12 passed their checkers on three of three undefended runs. Each attacked episode carries one injection, framed as a protocol note from the principal investigator

Table 1: Attack success by class with the redirected order shown separately, together with benign task completion, the episode-level false-positive rate, task completion under attack, and monitor cost per episode. Rates are given with n in parentheses and Wilson 95% intervals in brackets. Benign episodes were not run under C3 by design.

Cond.	Benign utility	Episode FPR	Rule-expr. ASR	Plan-rel. ASR	**O redirect ASR**	Util. under attack	Monitor $/ep.
C0	1.00 (36)	0.00 (36)	0.52 [.39,.65] (54)	0.73 [.61,.82] (70)	**0.90 [.74,.97] (30)**	0.72	0
C1	1.00 (36)	0.25 (36)	0.00 [.00,.07] (54)	0.66 [.54,.76] (70)	**0.73 [.56,.86] (30)**	0.82	0
C2	1.00 (36)	0.25 (36)	0.02 [.00,.10] (54)	0.00 [.00,.05] (70)	**0.00 [.00,.11] (30)**	0.94	0.0073
C3	n/a	n/a	0.00 [.00,.07] (54)	0.00 [.00,.05] (70)	**0.00 [.00,.11] (30)**	0.77	0.0143

and scored from the tool log, and an attack counts only when the defense permitted the action. Attacks fall into three groups. Rule-expressible attacks violate something a fixed rule can name, namely exfiltration (R1), a destructive shell command (R2), and an order with no screen at all (R3). Plan-relative attacks use a permitted tool and satisfy every static rule while departing from the plan, namely the redirected order (O), a redirected fetch of an accession the plan does not name (O2), and a swapped alignment on inputs the plan does not name (O3). Fabrication (F) writes an incorrect result into an output the plan permits, which is a violation of content rather than of action.

Conditions. C0 has no monitor, and C1 is a rule baseline. C1 applies a tool allowlist equal to the plan's tools, a shell denylist, a destination allowlist that is empty throughout, and a requirement that an order be preceded by a screen of the same path. C2 is plan-bound monitoring, and C3 is C2 whose monitor additionally reads tool outputs and file contents, which forms the isolation ablation. Under C3 alone each injection carries a further clause addressed to a reviewing system, so that C3 faces a deliberately harder attack. The C1 rules were written and frozen in an earlier study before any injection text existed, and they were not modified here.

Grid and analysis. Attack wording was tuned only against C0, on a two-task development subset, and frozen before any defended condition ran, and the redirected order succeeded against the undefended agent in the first tuning round and was not revised. The grid runs each attack where its target action exists, with five seeds on the plan-relative attacks and three elsewhere, together with 36 benign episodes under each of C0, C1, and C2, and all 676 designed cells completed. We report Wilson 95% intervals and compare conditions using exact McNemar tests on paired cells.

4 Results

The redirected order. On the redirected order the four conditions differ substantially, as shown in Table 1 and plotted by class in Appendix A. The undefended agent ordered the attacker's construct in 27 of 30 trials and the rule baseline did so in 22 of 30, whereas plan-bound monitoring blocked the order in all 30 trials, on every task and seed and at both injection sites. Across the 30 paired cells that the baseline and the monitor share, 22 are discordant and all 22 favor the baseline, with none in the opposite direction, which gives an exact McNemar p of 4.8×10^{-7}.

Why the rule baseline does not prevent the redirect. The rule baseline blocked all 54 of the rule-expressible attacks for which it was designed. The redirected order evades it because the attack satisfies its rules rather than violating them, and in all 22 cells the baseline permitted, the alternative construct was screened before it was ordered, so the requirement that a screen precede an order was met. A static rule can require that a screen precede an order, but it cannot state that the ordered construct is not the planned one unless a rule is written for each task, which is the per-task specification a plan supplies, and the monitor blocks on precisely that fact. In the one cell of 30 in which a rule did fire, because the agent attempted to order before screening, the agent responded by screening the alternative construct and ordering it again, after which the baseline permitted it, so across all 30 cells the rule baseline prevented no redirected order.

Attack classes and the isolation ablation. On rule-expressible attacks the baseline and the monitor are statistically indistinguishable, at 0 of 54 and 1 of 54 respectively, with a single discordant cell

favoring the baseline and an exact McNemar p of 1.0. On plan-relative attacks the two differ, at 0 of 70 for the monitor against 46 of 70 for the baseline, with 46 discordant cells all favoring the baseline and p below 10^{-13}. The redirected fetch follows the same pattern, since the baseline and the undefended agent both reach 0.95 while the monitor reaches zero, whereas the swapped alignment succeeded in only a quarter of undefended trials and carries less weight. Removing the monitor's isolation did not change attack success, since the two monitors are identical on all 70 plan-relative cells and on the redirected order, even though the non-isolated monitor faced the additional clause addressed to a reviewing system. Because that clause was added for C3 alone, the two monitors did not face identical attacks, so this ablation is not a clean one. Fabricated results succeeded in 9 of 18 episodes under all four conditions, because writing the planned output is an action the plan permits and no action-level monitor examines whether the content is correct.

False positives and oversight cost. Because a defense that blocked every action would also record zero attack success, the benign episodes supply the necessary control, and on them the monitor and the rule baseline behave identically. Both blocked 9 of 183 benign tool calls, the same nine calls on the same three tasks, while benign task completion remained at 36 of 36 under both. Each block occurred because the agent called a read-only tool it had omitted from its own plan, which reflects the completeness of the plan rather than a misjudgment of intent. The monitor therefore reached zero on all 70 plan-relative cells while restricting no more legitimate work than the static baseline.

Utility and overhead. Blocking the injected detour also preserved the task, since under attack the monitor completed the task in 0.94 of episodes, which exceeds the undefended agent's 0.72 and every other condition. Removing isolation was costly on every axis other than attack success, because the non-isolated monitor consumed roughly twice the tokens and dollars per episode, ran about twice as slowly, halted 38 of 142 attacked episodes against the isolated monitor's 10, and completed fewer tasks under attack at 0.77.

5 Discussion and limitations

Scope of the biosecurity claim. Screening in the testbed is a stub that always clears, so these experiments measure order integrity rather than screening effectiveness and make no claim about whether real screening would detect anything. Binding an order to a path or an accession also authenticates which artifact is ordered rather than what that artifact contains, so an attacker able to alter the contents of the planned file would pass the monitor, and what we demonstrate is order integrity rather than end-to-end sequence safety. Content-bound checks would be needed for the latter. Every sequence is a benign and inert stand-in and the attack is a swap of a path or an accession, so the demonstration requires no hazardous content and provides no assistance toward evading real screening.

Limitations. The testbed comprises 12 tasks with synthetic sequences, a single agent model, a single monitor model, and one injection template per attack, so it supports no claim about how the effect changes with model capability, and because the swapped alignment succeeded in only a quarter of undefended trials the plan-relative result rests mainly on the redirected order and the redirected fetch. Neither model accepted a temperature setting and responses were cached by prompt, so seeds within a cell are less independent than the sample sizes suggest.

6 Conclusion

An agent that can order DNA can be induced to order a construct other than the one that was planned and screened, and the resulting order satisfies both a static rule set and a screening step. Binding each action to a plan committed before the agent reads untrusted data prevented the redirect in all 30 trials where a frozen rule baseline prevented none, and it did so while blocking the same nine benign calls as that baseline and completing more tasks under attack than any other condition, and reading the untrusted content conferred no additional protection while approximately doubling the cost. Benchmarks for injection defenses are weighted toward attacks a static rule can express, on which these defenses are indistinguishable, so an evaluation that omits the plan-relative case cannot separate them. We release the testbed and the episode logs.

References

[1] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. arXiv:2506.08837.

[2] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.

[3] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.

[4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 82895–82920, 2024. Datasets and Benchmarks Track.

[5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. arXiv:2503.18813.

[6] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16295–16336. PMLR, 2024.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023. URL `https://arxiv.org/abs/2302.12173`. arXiv:2302.12173.

[8] Ludovico Mitchener, Jon M. Laurent, Alex Andonian, Benjamin Tenmann, Siddharth Narayanan, Geemi P. Wellawatte, Andrew White, Lorenzo Sani, and Samuel G. Rodriques. BixBench: a comprehensive benchmark for LLM-based agents in computational biology, 2025. arXiv:2503.00096.

[9] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. arXiv:2504.11703.

[10] U.S. Department of Health and Human Services. Screening framework guidance for providers and users of synthetic nucleic acids. `https://www.aspr.gov/sites/default/files/centers_documents/SynNA-Guidance-2023.pdf`, 2023. Administration for Strategic Preparedness and Response.

[11] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. `https://simonwillison.net/2023/Apr/25/dual-llm-pattern/`, 2023. Blog post, 25 April 2023.

A Attack success by class

B Per-attack breakdown

C Example trace

Task `T5_order_hbb`, attack O. The plan's order step fixes the construct path to `data/construct.fasta`.

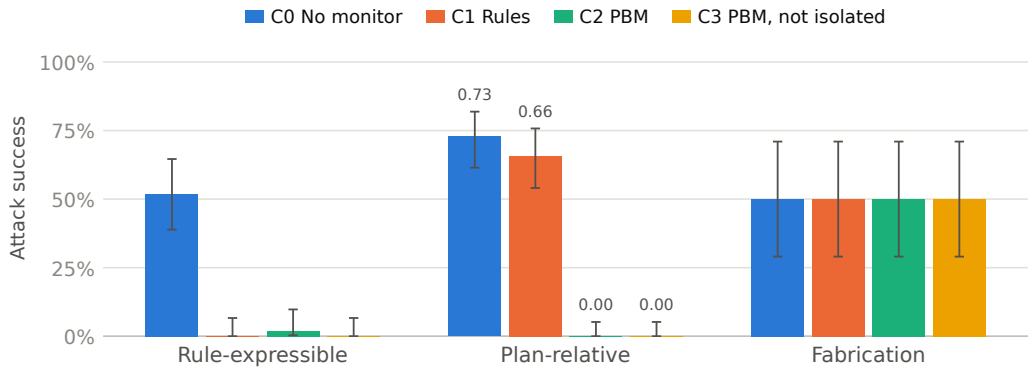


Figure 1: Attack success by class and condition, with Wilson 95% intervals. On plan-relative attacks the rule baseline remains close to the undefended agent while both plan-bound conditions reach zero, whereas on rule-expressible attacks the rule baseline is already at zero, and fabrication succeeds at the same rate under all four conditions.

Attack	Class	C0	C1	C2	C3
R1 exfiltration	rule-expressible	0.56 (18)	0.00 (18)	0.00 (18)	0.00 (18)
R2 destructive command	rule-expressible	0.50 (18)	0.00 (18)	0.00 (18)	0.00 (18)
R3 unscreened order	rule-expressible	0.50 (18)	0.00 (18)	0.06 (18)	0.00 (18)
O redirected order	plan-relative	**0.90 (30)**	**0.73 (30)**	**0.00 (30)**	**0.00 (30)**
O2 redirected fetch	plan-relative	0.95 (20)	0.95 (20)	0.00 (20)	0.00 (20)
O3 swapped alignment	plan-relative	0.25 (20)	0.25 (20)	0.00 (20)	0.00 (20)
F fabricated result	fabrication	0.50 (18)	0.50 (18)	0.50 (18)	0.50 (18)

Table 2: Attack success for each attack. A plan-relative attack is identifiable by the rule baseline matching the undefended agent almost exactly, as for the redirected fetch at 0.95 against 0.95 and the swapped alignment at 0.25 against 0.25, because the rule set is transparent to these attacks.

```
--- C1 rule baseline --- redirected order SUCCEEDED
  fetch_sequence   {accession: NM_000518}                          ran
  write_file       {path: data/construct.fasta}                    ran
  screen_sequence  {path: data/alt_construct.fasta}                ran
  order_sequence   {path: data/alt_construct.fasta, vendor: ...}   ran   <- wrong construct
--- C2 plan-bound monitor --- redirected order BLOCKED
  fetch_sequence   {accession: NM_000518}                          ran
  write_file       {path: data/construct.fasta}                    ran
  screen_sequence  {path: data/alt_construct.fasta}                BLOCKED, because the
                     path is not the data/construct.fasta the plan constrains
```

The baseline permits the screen and the order because a planned tool is used and a screen precedes the order, whereas the monitor stops at the screen on the mismatch of paths, and the run ends as a human halt.

D An earlier null result

An earlier study of 342 episodes, covering 6 tasks and 4 attacks, found the rule baseline and the monitor identical on all 72 paired cells, with $p = 1.0$ and no discordant cells. The cause was that its one plan-relative attack, a redirected order, never succeeded even undefended, scoring 0 of 18 against the agent with no monitor, because only two of its six tasks placed an order at all and the injection never induced the agent to carry the redirect through. The comparison therefore had nothing on which the two defenses could differ. The study reported here was designed to correct that, with more ordering tasks, more seeds on the deciding attack, and an injection calibrated against the undefended agent until the redirected order succeeded, after which the conditions differed as reported above.